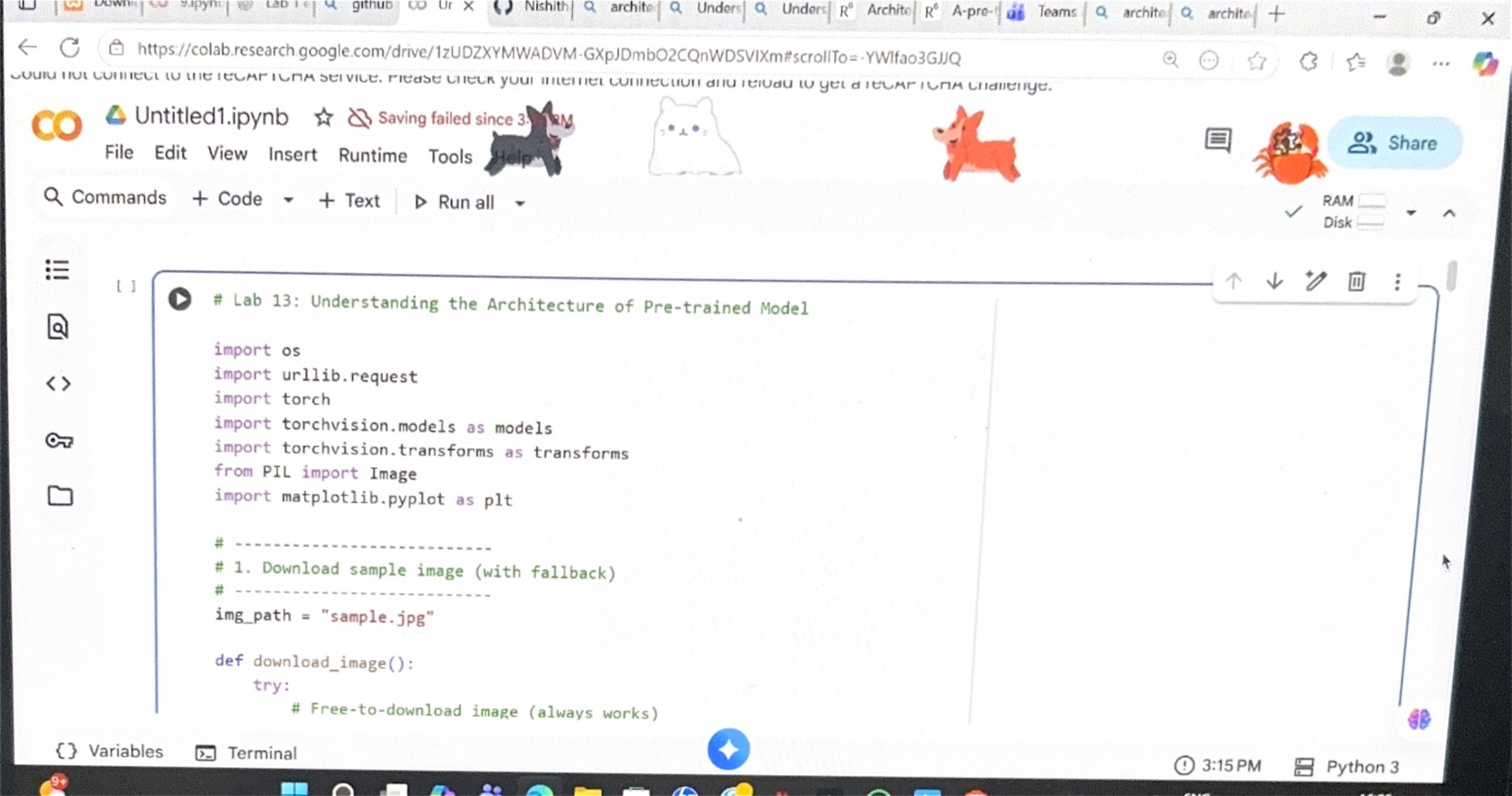




Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.



Untitled1.ipynb  Saving failed since 3:54 PM 

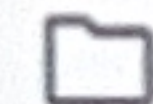
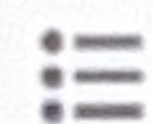
File Edit View Insert Runtime Tools Help



Share

Commands + Code + Text ▶ Run all

✓ RAM Disk



```
[ ]
# Count parameters
total_params = sum(p.numel() for p in model.parameters())
trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"\nTotal Parameters: {total_params:,}")
print(f"Trainable Parameters: {trainable_params:,}")

# -----
# 3. Load and Preprocess Image
# -----
transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])
])

img = Image.open(img_path).convert('RGB')
input_tensor = transform(img).unsqueeze(0)
```



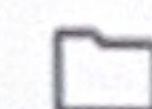
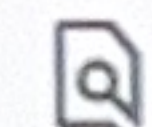
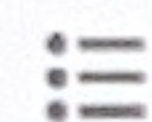
Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.



Untitled1.ipynb ☆ Saving failed since 3:54 PM

File Edit View Insert Runtime Tools Help

Commands + Code + Text ▶ Run all



[]

```
▶ # Display input image
plt.imshow(img)
plt.title("Input Image")
plt.axis("off")
plt.show()

# -----
# 4. Forward Pass & Extract Features
# -----

activation = {}

def get_activation(name):
    def hook(model, input, output):
        activation[name] = output.detach()
    return hook

# Register hooks
model.layer1.register_forward_hook(get_activation('layer1'))
model.layer2.register_forward_hook(get_activation('layer2'))
... ..
```



Share

✓ RAM Disk



could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.




Untitled1.ipynb
☆
⛔
Session failed since 3:54 PM

File
Edit
View
Insert
Runtime
Tools
Help

Share

Commands
+ Code
+ Text
▶ Run all

✓
RAM
Disk

```
[ ] ▶ model.layer3.register_forward_hook(get_activation('layer3'))
model.layer4.register_forward_hook(get_activation('layer4'))

# Forward pass
with torch.no_grad():
    output = model(input_tensor)

# Print feature map shapes
print("\n--- Intermediate Feature Map Shapes ---")
for name, act in activation.items():
    print(f"{name}: {tuple(act.shape)}")

# -----
# 5. Visualize Feature Maps
# -----

layer_name = 'layer1'
feature_map = activation[layer_name].squeeze(0)

plt.figure(figsize=(12, 6))
for i in range(6):
```

←↻🔒https://colab.research.google.com/drive/1zUDZXYMWADV...GxpJDmbO2CQnWDSVIXm#scrollTo=-YWlfao3GJJQ🔍🗨️⭐⚙️🌟👤⋮🌈

could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.

COUntitled1.ipynb⭐🔄Saving failed since 3:54 PM🐱🍁

FileEditViewInsertRuntimeToolsHelp

🔍Commands+ Code+ Text▶ Run all

☰🔍🔑📁

▶[]▶

```
plt.subplot(2, 3, i + 1)
fm = feature_map[i].cpu()
fm = (fm - fm.min()) / (fm.max() - fm.min())
plt.imshow(fm, cmap='viridis')
plt.axis('off')
plt.title(f'{layer_name} - Channel {i}')
plt.tight_layout()
plt.show()

# -----
# 6. Show Top-5 Predictions
# -----
probabilities = torch.nn.functional.softmax(output[0], dim=0)
top5_prob, top5_catid = torch.topk(probabilities, 5)

# Load ImageNet labels if available
try:
    with open("imagenet_classes.txt") as f:
        categories = [line.strip() for line in f.readlines()]
        # -----
```

⬆️⬆️✎️🗑️⋮

📁Variables📄Terminal

⚙️3:15 PM🖨️Python 3

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.



Untitled1.ipynb ☆ Saving failed since 3:54 PM

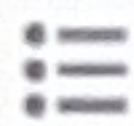


Share

File Edit View Insert Runtime Tools Help

Commands + Code + Text ▶ Run all

RAM Disk



```
[ ] categories = [f"Class {i}" for i in range(1000)]

print("\n--- Top 5 Predictions ---")
for i in range(5):
    print(f"{categories[top5_catid[i]]}: {top5_prob[i].item()*100:.2f}%")
```



✓ Downloaded sample.jpg successfully!

```
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated
warnings.warn(
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or
warnings.warn(msg)
Downloading: "https://download.pytorch.org/models/resnet50-0676ba61.pth" to /root/.cache/torch/hub/checkpoints/resnet50-0676
100%|██████████| 97.8M/97.8M [00:00<00:00, 157MB/s]
```

--- Model Architecture ---

ResNet(

{ } Variables

Terminal



3:15 PM

Python 3

Download9.ipynbLab 1githubUr XNishitharchiteUndersUndersR⁰ArchiteR⁰A-pre-1Teamsarchitearchite

https://colab.research.google.com/drive/1zUDZXYMWADV...GxpJDmbO2CQnWDSVIXm#scrollTo=-YWlfao3GJJQ

COOLab

FileEditViewInsertRuntimeToolsHelp

Commands+ Code+ Text▶ Run all

RAMDisk

Share

--- Model Architecture ---

ResNet(
 (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)
 (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
 (relu): ReLU(inplace=True)
 (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=False)
 (layer1): Sequential(
 (0): Bottleneck(
 (conv1): Conv2d(64, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
 (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
 (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
 (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
 (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
 (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
 (relu): ReLU(inplace=True)
 (downsample): Sequential(
 (0): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
 (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
)
)
)
)

VariablesTerminal

3:15 PMPython 3

Downlo9.ipynbLab 1githubUr XNishitharchiteUndersUndersR⁰ ArchiteR⁰ A-pre-1Teamsarchitearchite

https://colab.research.google.com/drive/1zUDZXYMWADV...#scrollTo=-YWI...fao3GJJQ

COULD NOT CONNECT TO THE RECAPTCHA SERVICE. PLEASE CHECK YOUR INTERNET CONNECTION AND RELOAD TO GET A RECAPTCHA CHALLENGE.

COUntitled1.ipynb★Saving failed since 3:54 PM

FileEditViewInsertRuntimeToolsHelp

QCommands+ Code+ Text▶Run all

RAMDisk

↑↓✎🗑⋮

☰🔍🔑📁

```
(0): Bottleneck(
  (conv1): Conv2d(256, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
  (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU(inplace=True)
  (downsample): Sequential(
    (0): Conv2d(256, 512, kernel_size=(1, 1), stride=(2, 2), bias=False)
    (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  )
)
(1): Bottleneck(
  (conv1): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
  (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
```

🌟

{ } VariablesTerminal

⚠️ 3:15 PMPython 3

COULD NOT CONNECT TO THE RECAPTCHA SERVICE. PLEASE CHECK YOUR INTERNET CONNECTION AND RELOAD TO GET A RECAPTCHA CHALLENGE.



```
(0): Bottleneck(
  (conv1): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
  (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU(inplace=True)
)
(layer3): Sequential(
  (0): Bottleneck(
    (conv1): Conv2d(512, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
    (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU(inplace=True)
    (downsample): Sequential(
      (0): Conv2d(512, 1024, kernel_size=(1, 1), stride=(2, 2), bias=False)
```





[]

```
import matplotlib.pyplot as plt

layer_names = ['layer1', 'layer2', 'layer3', 'layer4']
channels = [256, 512, 1024, 2048]

plt.figure(figsize=(7,4))
plt.bar(layer_names, channels, color='skyblue')
plt.title('Feature Map Channels per Layer (ResNet50)')
plt.xlabel('Layer')
plt.ylabel('Number of Channels')
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()

classes = ['Class 210', 'Class 178', 'Class 208', 'Class 206', 'Class 209']
confidence = [85.62, 6.93, 3.82, 1.46, 1.38]

plt.figure(figsize=(7,4))
plt.barh(classes, confidence, color='orange')
plt.xlabel('Confidence (%)')
plt.title('Top-5 Model Predictions')
```

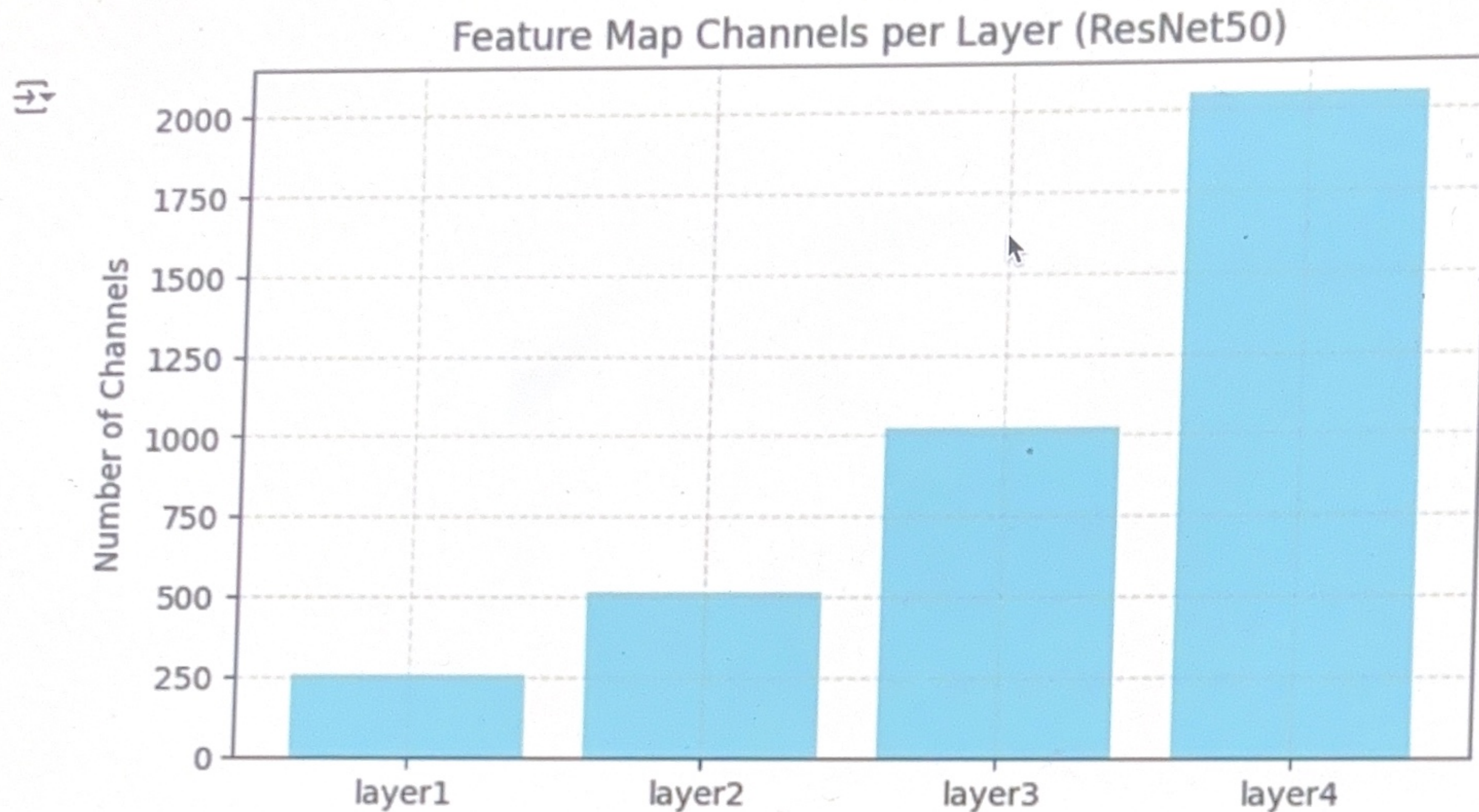
Untitled1.ipynb ☆ Saving failed since 3:54 PM
File Edit View Insert Runtime Tools Help

Commands + Code + Text ▶ Run all

Share

RAM
Disk

↑ ↓ ✎ 🗑 ⋮





13. Understanding the architecture of pre-trained model

Aim:-

To explore & understand the architecture of pre-trained deep learning model used for image classification.

Objective:-

- To load & visualize a pre-trained model from a deep learning library.
- To analyse the layer wise structure, parameters and activation functions.
- To understand the concept of transfer learning using pre-trained weights.
- To visualize feature extraction from intermediate layer.

Pseudocode:-

1. Import necessary libraries.
2. Load pretrained model with imagenet weight.
3. Display model summary.
4. Observe:
 - input & output shapes
 - layer names, types and number of parameters.
5. visualize the architecture using:
`plot_model(model, show_shapes=True)`
6. load a sample input image & preprocess it for the model.
7. pass the image through the model & get prediction.

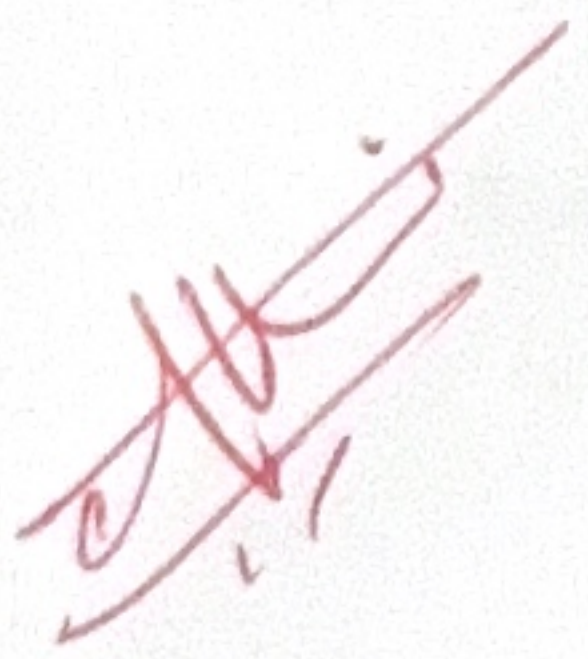
Observation:-

- The pre-trained model contains millions of parameter spread across convolutional & dense layer.

- Early layer edges & texture, while deeper layer capture high-level features, such as object parts.
- The use of pretrained weights drastically reduces training time & improve model accuracy for new task.

Result:-

Successfully explored and understood the architecture of pre-trained model.



Output:-

Total parameters : 25,557,032

Tranable parameter: 25,557,032.

intermediate feature map shapes:

layer 1: (1, 256, 56, 56)

layer 2: (1, 512, 28, 28)

layer 3: (1, 1024, 14, 14)

layer 4: (1, 2048, 7, 7)

TOP 5 prediction.

class 210 : 65.62%

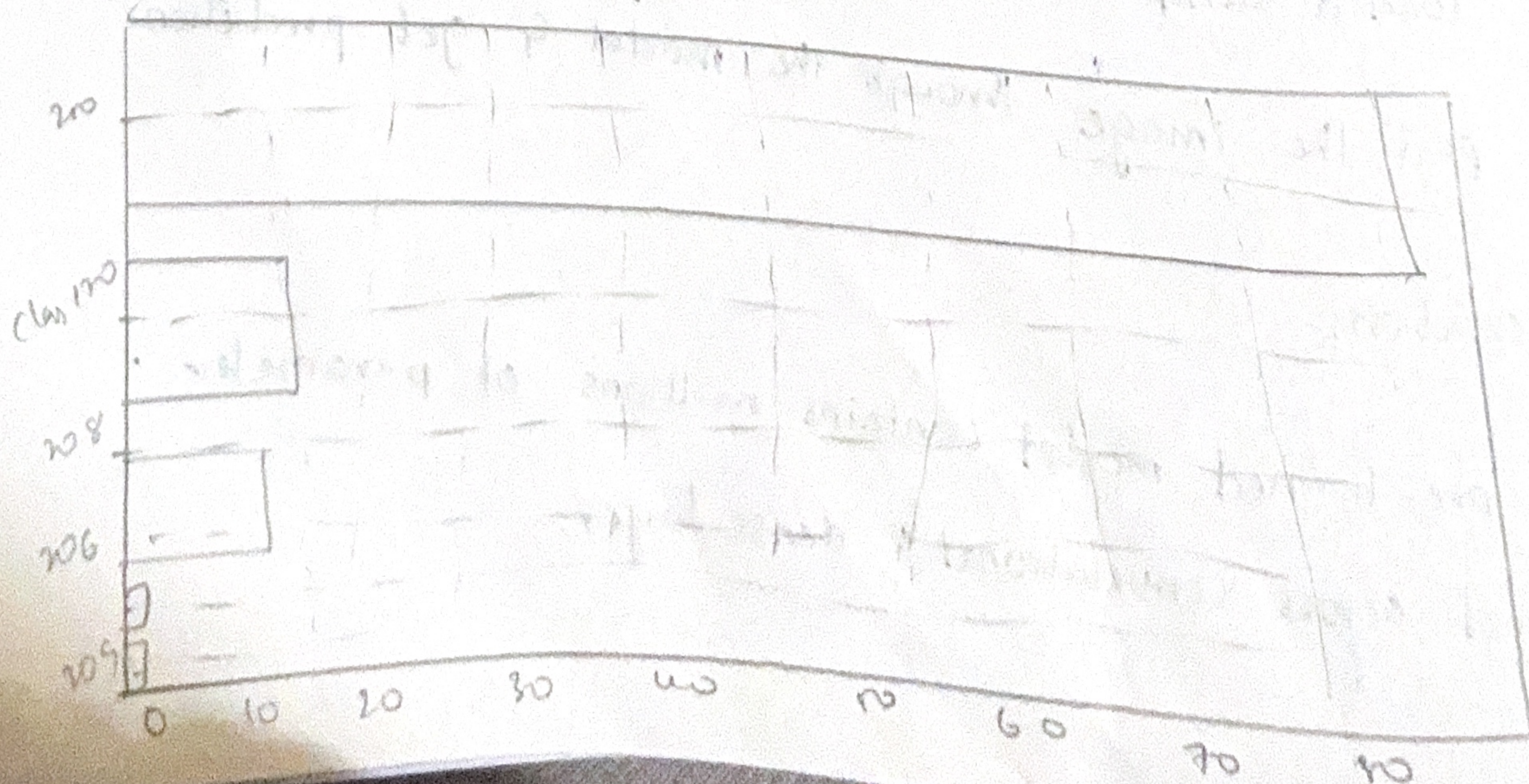
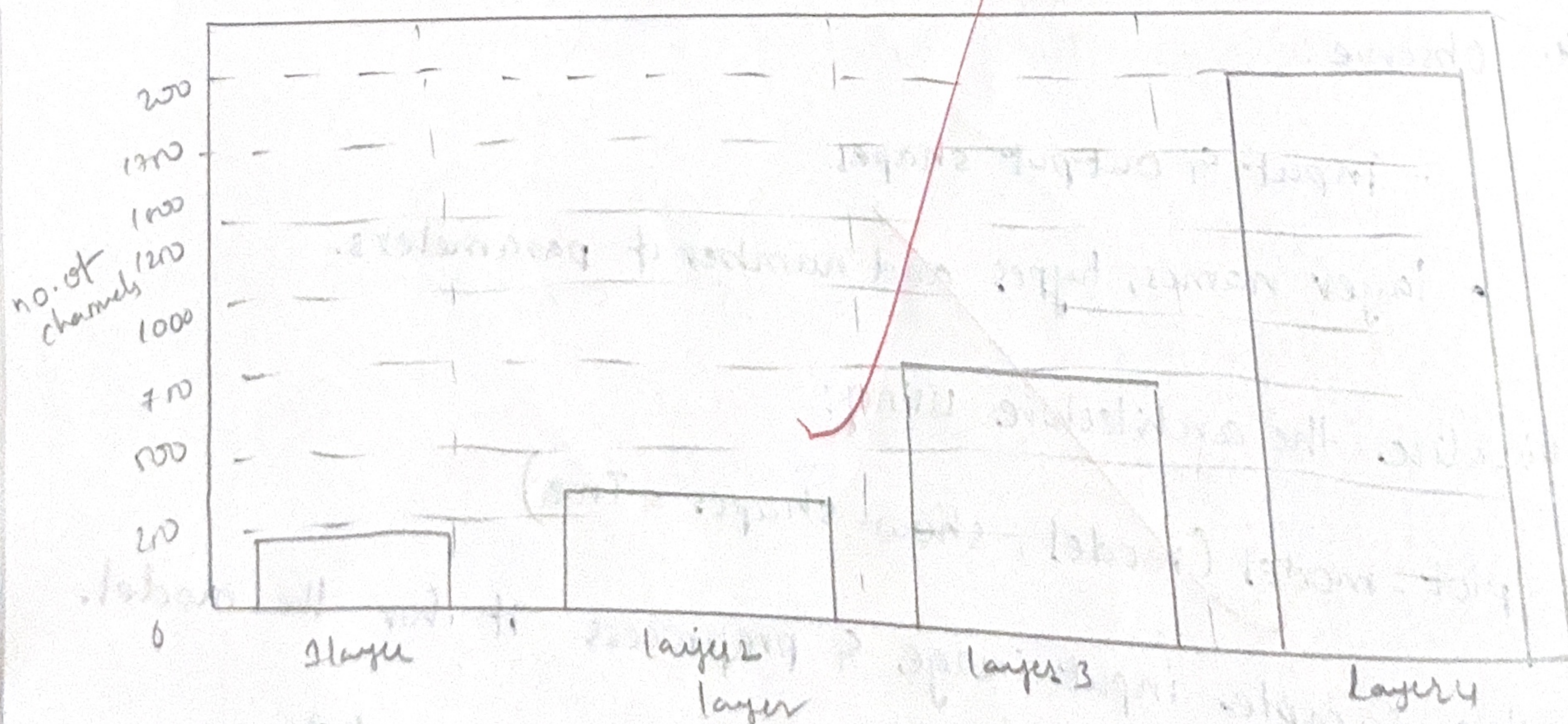
class 178 : 6.93%

class 202 : 3.82%

class 206 : 1.46%

class 209 : 1.38%

feature map channel per layer (resnet 50)



- Early layer edges & high-level features.
- The rise of pretrain time & improve model

Result:-

Successfully e
pre-trained model

Architecture of pre-trained Model:-

